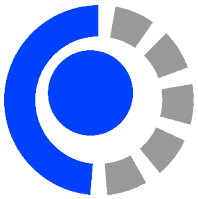


Universidad Nacional del Comahue

Facultad de Informática



Trabajo Práctico Obligatorio
Conceptos Avanzados de Lenguajes de Programación

Diaz, Guillermo	FAI-3197
Quintero, Kevin	FAI-2771
Rodriguez, Jeremías	FAI-2543
Sepúlveda, Luciano	Leg. 72662

Neuquén Argentina

2025

1. Ejercicio 1: En el contexto de subrutinas y pasaje de parametros, defina los conceptos

1.1. Tipo de pasaje por valor

Cuando un parámetro se transmite por valor, el subprograma recibe una copia independiente del valor del parámetro real. Esto significa que cualquier modificación que se realice sobre el parámetro formal dentro del subprograma no afecta al valor original en el programa principal. En otras palabras, el parámetro formal actúa como una variable local que contiene una copia del dato, de modo que los cambios realizados sobre él se pierden una vez que finaliza la ejecución del subprograma. Este mecanismo se utiliza comúnmente cuando se desea preservar la integridad de los datos originales.

1.2. Tipo de pasaje por resultado

Un parámetro pasado por resultado se utiliza únicamente para devolver un valor desde un subprograma. El valor inicial del parámetro real no tiene influencia alguna y no puede ser empleado dentro del subprograma. El parámetro formal actúa como una variable local sin valor inicial (salvo que se le asigne una inicialización por defecto, como ocurre con otras variables locales). Al finalizar la ejecución del subprograma, el valor final del parámetro formal se asigna al parámetro real, de forma similar a lo que ocurre en la llamada por valor-resultado.

1.3. Tipo de pasaje por referencia

La llamada por referencia (call by reference) es, quizá, el mecanismo de transmisión de parámetros más común. Transmitir un objeto de datos mediante llamada por referencia significa que se proporciona al subprograma un puntero a la ubicación del objeto de datos, es decir, el valor que representa su dirección en memoria. El objeto de datos no cambia de posición en memoria. Al inicio de la ejecución del subprograma, los punteros de los parámetros reales se utilizan para inicializar las ubicaciones de almacenamiento local de los parámetros formales.

El paso de parámetros por referencia ocurre en dos etapas principales:

1. En el subprograma que realiza la llamada, cada expresión de parámetro real se evalúa para obtener un puntero al objeto de datos correspondiente.
2. Luego, se almacena una lista de estos punteros en un área de memoria común accesible también por el subprograma llamado (a menudo en un conjunto de registros o en la pila del programa).
3. A continuación, el control se transfiere al subprograma llamado.

En el subprograma llamado, la lista de punteros a los parámetros reales se utiliza para acceder a los valores correspondientes de dichos parámetros. Durante la ejecución del subprograma, las referencias a los nombres de los parámetros formales se tratan como referencias a variables locales comunes (aunque puede existir una selección implícita mediante punteros). Al finalizar la ejecución del subprograma, los resultados se devuelven al programa que realizó la llamada a través de los objetos de datos de los parámetros reales.

1.4. Tipo de pasaje por nombre

El paso de parámetros por nombre consiste en un modelo de transmisión donde la llamada a un subprograma se interpreta como una sustitución textual de su cuerpo, reemplazando cada parámetro formal por el parámetro real correspondiente.

En este esquema, los parámetros reales no se evalúan al momento de la llamada, sino cada vez que son utilizados dentro del subprograma. De esta forma, el subprograma decide cuándo se realiza la evaluación.

Aunque este método tiene interés teórico y fue característico del lenguaje ALGOL, su alto costo de implementación lo vuelve poco práctico para subprogramas definidos por el programador.

La regla fundamental establece que el parámetro real debe sustituir todas las apariciones del parámetro formal en el cuerpo del subprograma antes de su ejecución. Sin embargo, esta sustitución debe mantener el entorno de referencia del programa que realiza la llamada, lo que puede generar ambigüedades semánticas si existen variables con nombres coincidentes.

1.5. Ejemplique con programas donde el resultado sea diferente

1.5.1. Ejemplo pasaje por valor/resultado y por referencia

```
program Main;
    var
      j, m, i: integer;

    procedure Recibe(x: integer; y: integer);
    begin
      m := m + 1 + y;
      x := i + x + j;
      y := m - 1;
      write(x, ' ', y, ' ', i, ' ', j, ' ', m);
      writeln;
    end;

    procedure Dos;
    var
      m: integer;
    begin
      m := 5;
      Recibe(i, j);
      write(i, ' ', j, ' ', m);
      writeln;
    end;

    begin
      m := 2;
      i := 1;
      j := 3;
      Dos;
      write(i, ' ', j, ' ', m);
      writeln;
    end.
```

TRAZAS PASANDO PARÁMETROS POR REFERENCIA

Instrucción	j	m	i	Llamadas	Salida
$m := 2;$		2			
$i := 1;$			1		
$j := 3;$	3				
$Dos;$	Modificaciones por Dos (Ninguna) ——— Modificaciones por Recibe $6 - 1 = 5$	Modificaciones por Recibe $2 + 1 + 3 = 6$	Modificaciones por Dos (Ninguna) ——— Modificaciones por Recibe $1 + 1 + 3 = 5$	Dos	
$write(i,\ j,\ m);$					i, j, m $5, 5, 6$

Cuadro 1: Traza del Main

Instrucción	$m \rightarrow$ Local	Llamadas	Salida
$m := 5;$	5		
$Recibe(i, j);$	Modificaciones por Recibe $5 + 1 + 3 = 9$	$Recibe(i, j)$	
$write(i,\ j,\ m);$			i, j, m $5, 5, 9$

Cuadro 2: Traza del Dos

Instrucción	$x \rightarrow$ referencia a i	$y \rightarrow$ referencia a j	Salida
$m := m + 1 + y;$	1	2	
$x := i + x + j;$	$1 + 1 + 3 = 5$		
$y := m - 1;$		$6 - 1 = 5$	
$write(x,\ y,\ i,\ j,\ m);$			x, y, i, j, m $5, 5, 5, 5, 6$

Cuadro 3: Traza del Recibe

TRAZAS PASANDO PARÁMETROS POR VALOR

Instrucción	j	m	i	Llamadas	Salida
$m := 2;$		2			
$i := 1;$			1		
$j := 3;$	3				
$Dos;$		Modificaciones por Recibe $2 + 1 + 3 = 6$		Dos	
$write(i,\ j,\ m);$					i, j, m $1, 3, 6$

Cuadro 4: Traza del Main por valor

Instrucción	m	Llamadas	Salida
$m := 5;$	5		
$Recibe(i, j);$		$Recibe(i, j) = Recibe(1, 3)$	
$write(i,\ j,\ m);$			i, j, m $1, 3, 5$

Cuadro 5: Traza del Dos por valor

Instrucción	x	y	Salida
$m := m + 1 + y;$	1	3	
$x := i + x + j;$	$1 + 1 + 3 = 5$		
$y := m - 1;$		$6 - 1 = 5$	
$write(x,\ y,\ i,\ j,\ m);$			x, y, i, j, m $5, 5, 1, 3, 6$

Cuadro 6: Traza del Recibe por valor

El mecanismo de implementación es similar al modelo de llamada por referencia, con las siguientes diferencias:

- Al invocar un subprograma, un parámetro pasado por referencia transmite un puntero al objeto real, mientras que un parámetro pasado por valor transmite una copia del valor.
- Dentro del subprograma, un parámetro pasado por referencia usa el valor almacenado en el parámetro formal para acceder directamente al dato original. En cambio, en la llamada por valor, el parámetro formal contiene una copia del valor y no tiene acceso al objeto real.

De esta forma, con la llamada por referencia se crea un alias del parámetro real (ambos apuntan al mismo dato), mientras que con la llamada por valor no existe tal vínculo.

Una vez que un parámetro se pasa por valor, el parámetro formal no puede modificar el valor del parámetro real. Cualquier cambio hecho en el parámetro formal durante la ejecución del subprograma se pierde al finalizar la ejecución del mismo.

1.5.2. Ejemplo pasaje por nombre y por referencia

```
procedure P(x, y);
begin
    x := x + 1;
    y := y + 1;
end;

begin
    i := 1;
    A[1] := 10;
    A[2] := 20;

    P(i, A[i]);
end;
```

TRAZAS PASANDO PARÁMETROS POR REFERENCIA

Instrucción	i	A[1]	A[2]
$i := 1$	1	–	–
$A[1] := 10$	1	10	–
$A[2] := 20$	1	10	20
$P(i, A[i])$	1	10	20
$P(i, A[i])$	2	11	20

Cuadro 7: Traza Main por referencia

Instrucción en P	Acción efectiva	i	A[1]	A[2]
$x := x + 1$	$i := i + 1$	2	10	20
$y := y + 1$	$A[1] := A[1] + 1$	2	11	20
Retorna a main	–	2	11	20

Cuadro 8: Traza de P por referencia

TRAZAS PASANDO PARÁMETROS POR NOMBRE

Instrucción	i	A[1]	A[2]
$i := 1$	1	–	–
$A[1] := 10$	1	10	–
$A[2] := 20$	1	10	20
$P(i, A[i])$	1	10	20

Cuadro 9: Traza del Recibe por valor

Instrucción en P	Sustitución efectiva	i	A[1]	A[2]
$x := x + 1$	$i := i + 1$	2	10	10
$y := y + 1$	$A[i] := A[i] + 1$	2	10	21
Retorna a main	$-$	2	10	21

Cuadro 10: Traza del Recibe por valor

Debido a que al pasaje por referencia primero se realiza la evaluación, el llamado a la función P resulta en P(i, A[1]). Mientras que, en el pasaje por nombre la evaluación es tardía o atrasada hasta el momento de su uso. Por lo tanto, la llamada resulta en P(i, A[i]), reemplazando en el código todas las apariciones de X por i, e Y por A[i].

2. Ejercicio 2

La Figura 1 muestra la estructura de la **pila de activación** del programa principal correspondiente al código en Ada. En la parte inferior se encuentra el **Activation Record Instance (ARI)** del subprograma principal `Subprogramas_Ada`, el cual contiene las variables locales declaradas dentro de él. Estas incluyen el arreglo `numero`, cuyas cinco posiciones se almacenan de forma secuencial, y la variable compuesta `estudiante`, representada con sus tres campos: `id`, `nombre` y `promedio`.

Por encima del ARI principal se apilan los registros de activación de los subprogramas invocados por el programa. Cada bloque identificado (por ejemplo, `Imprimir_Estudiante`, `Modificar_Estudiante` o `Procesar_Arreglo`) representa una llamada activa que ocupa espacio temporal en la pila durante su ejecución.

Cabe destacar que estos ARI se van creando y eliminando dinámicamente a medida que los subprogramas son llamados y retornan su control al programa principal. De esta manera, la pila refleja el orden de activación de los subprogramas y el estado actual de ejecución, donde el tope de la pila indica el subprograma que se encuentra ejecutándose en ese momento.

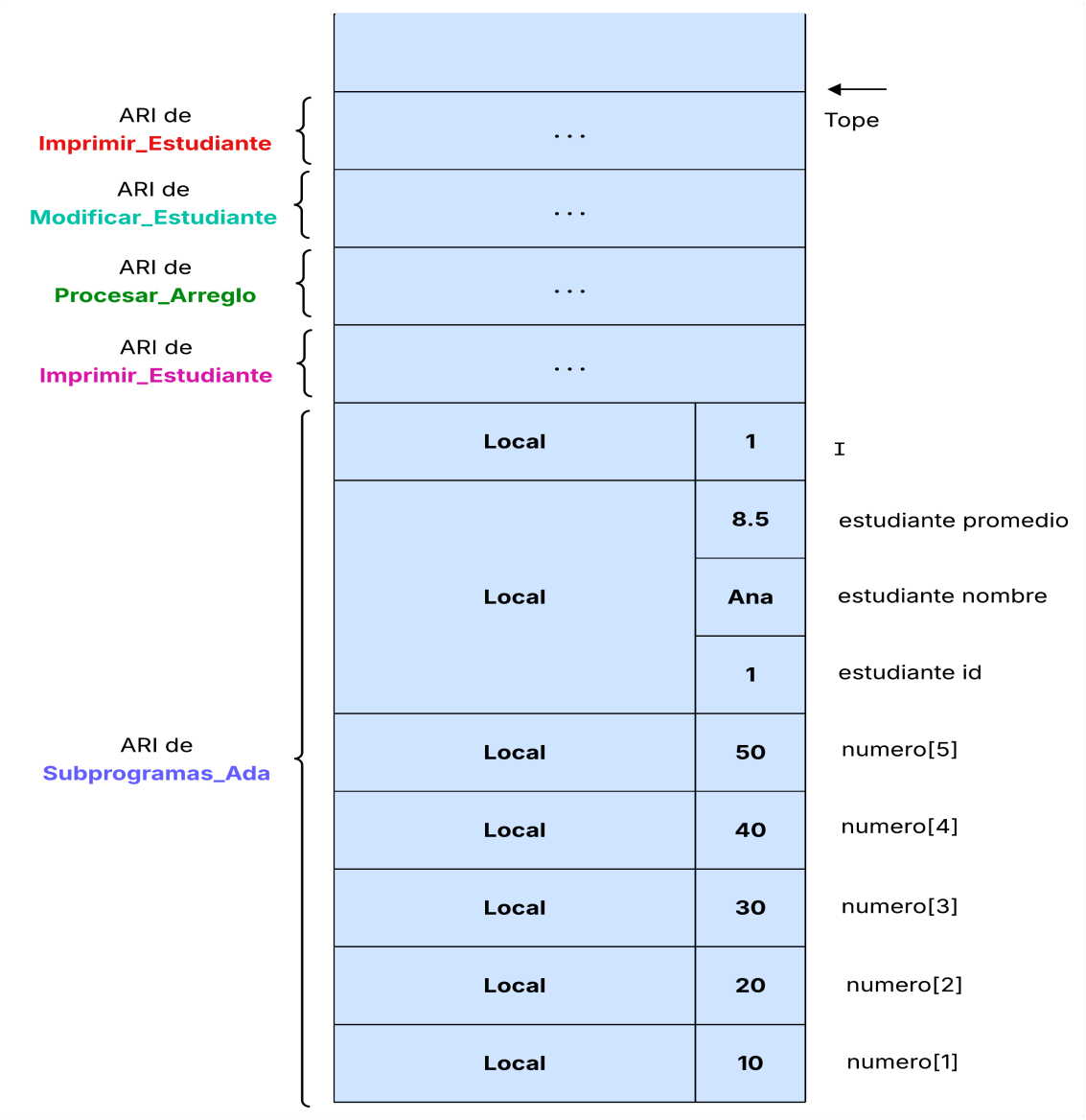


Figura 1: Pila de los registros de activación del programa principal.

La Figura 2 muestra una vista detallada del segmento de pila correspondiente al **Activation Record Instance (ARI)** del subprograma `Imprimir_Estudiante`. En la parte inferior del registro se encuentran los **enlaces estático y dinámico**, junto con la **dirección de retorno** hacia el ARI del programa principal `Subprogramas_Ada`.

Por encima de estos elementos se ubican los **parámetros del procedimiento**, en este caso la variable `estudiante`, pasada por el programa principal. El procedimiento utiliza este parámetro únicamente para lectura, ya que está declarado con el modo `in`. Por lo tanto, el compilador implementa el paso **por valor**, evitando modificaciones sobre la variable original.

Este procedimiento es invocado tanto al inicio como al final de la ejecución del programa principal. En la figura se representa la primera llamada, donde el valor del campo `promedio` del estudiante es `8.5`. En la segunda invocación, la estructura de la pila sería similar, pero los valores del parámetro cambiarían (por ejemplo, el campo `promedio` pasaría de `8.5` a `9.0`).

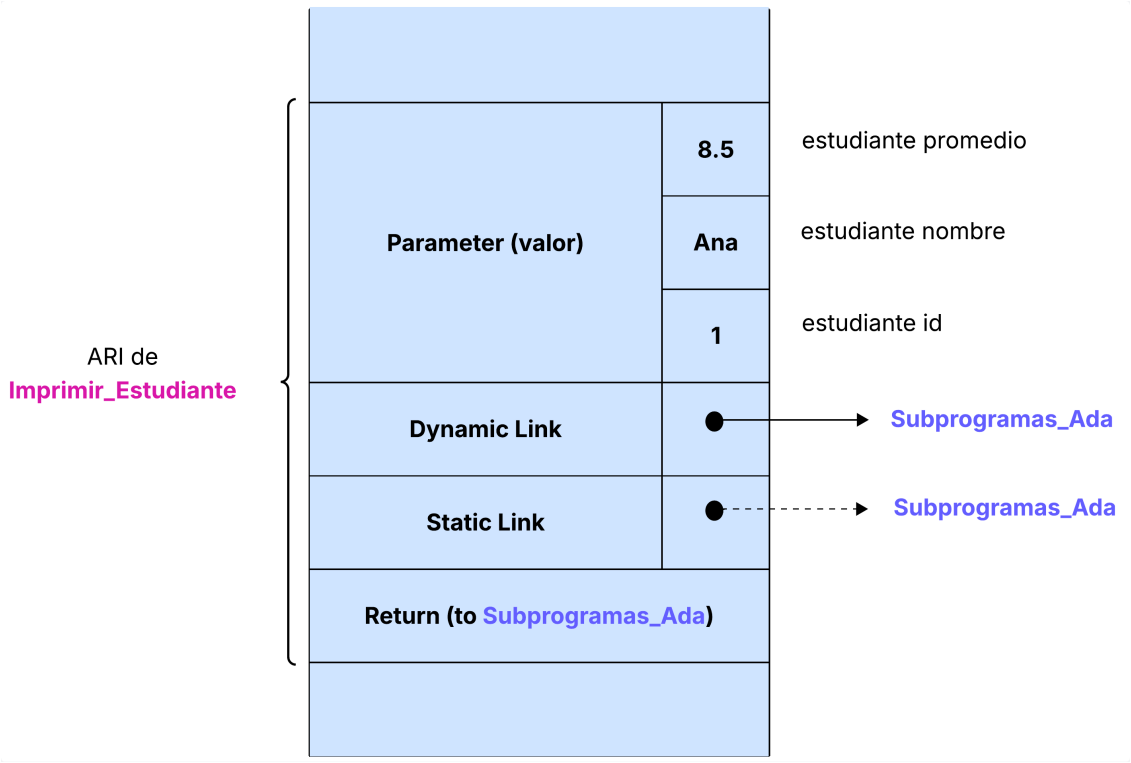


Figura 2: Pila de los registros de activación del procedimiento `Imprimir_Estudiante`.

La Figura 3 muestra la expansión de la pila correspondiente al **Activation Record Instance (ARI)** del subprograma `Procesar_Arreglo`. En este registro se observa que, además de los **enlaces estático y dinámico** y la **dirección de retorno** hacia el programa principal, el procedimiento contiene un parámetro por referencia (`Arr`) que apunta al arreglo original, y una variable local `I` utilizada como iterador para recorrer sus elementos.

Durante la ejecución del procedimiento se realizan múltiples llamadas al subprograma `Multiplicar_Elemento`, una por cada posición del arreglo. En la figura se ilustran dos de las cinco llamadas efectivas, mostrando cómo cada instancia recibe como parámetro una referencia a la posición del arreglo correspondiente al valor actual de `I`. De esta forma, `Multiplicar_Elemento` puede modificar directamente el contenido del arreglo original, reflejando los cambios sin necesidad de copiar los valores.

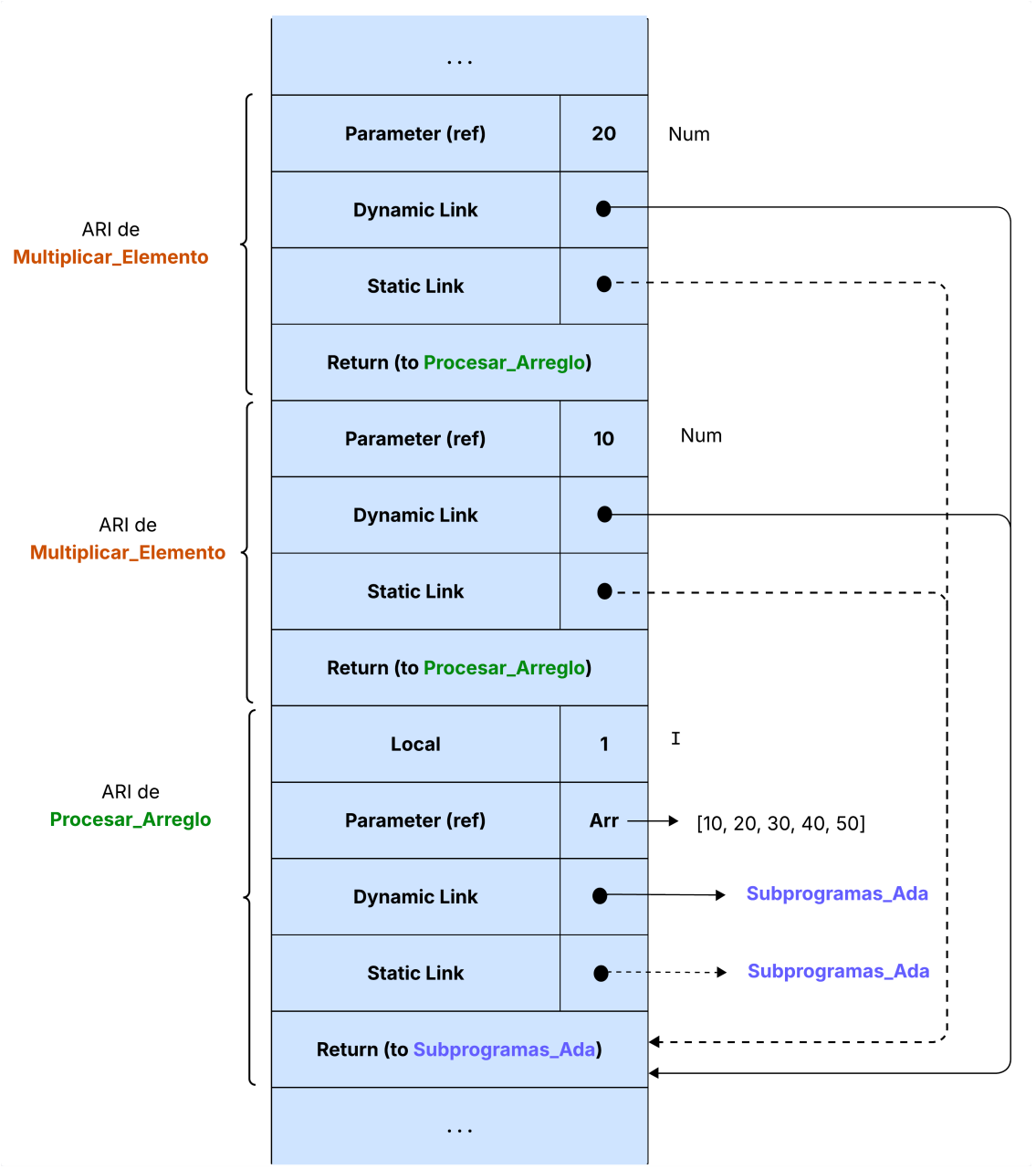


Figura 3: Pila de los registros de activación del procedimiento `Procesar_Arreglo`.

La Figura 4 muestra el **Activation Record Instance (ARI)** correspondiente al procedimiento `Modificar_Estudiante`. Al igual que en los casos anteriores, este registro de activación contiene los **enlaces estático y dinámico** que apuntan al ARI del procedimiento que lo invocó (`Subprogramas_Ada`), así como la **dirección de retorno** al mismo.

En la parte superior del ARI se encuentran los **parámetros por referencia** del procedimiento, que representan los datos del estudiante: `id`, `nombre` y `promedio`. Durante la ejecución, `Modificar_Estudiante` accede y modifica estos valores, y una vez finalizada su tarea, retorna el control al procedimiento invocador.

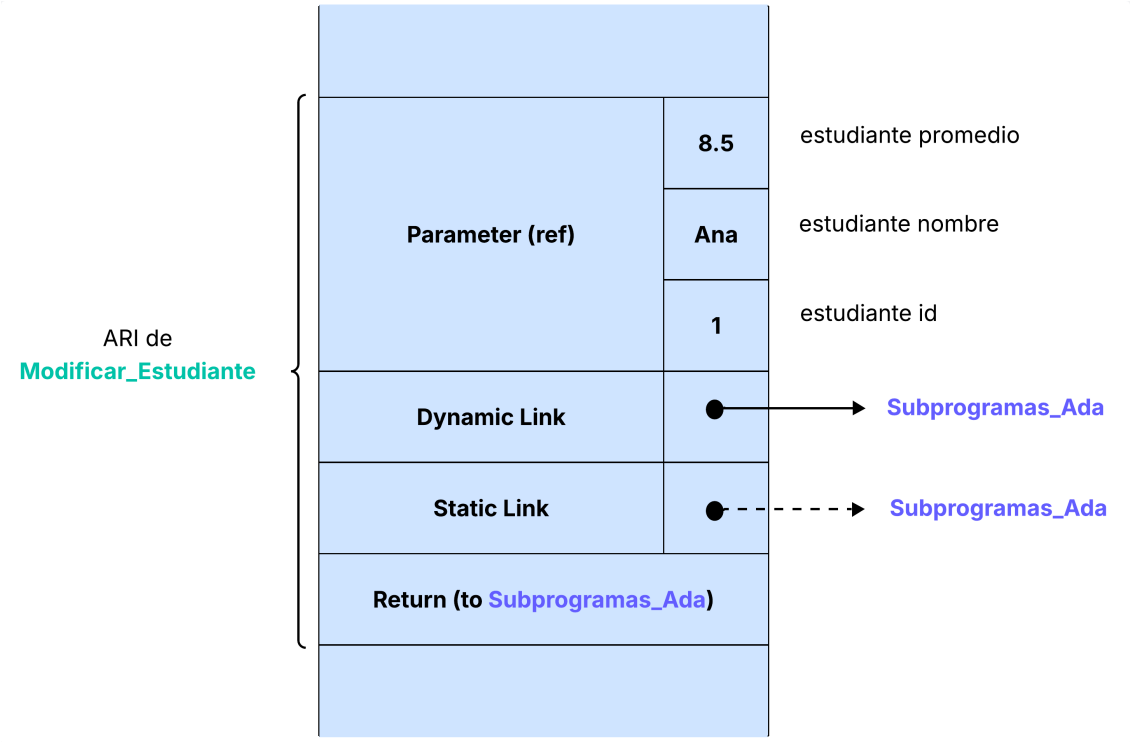


Figura 4: Pila de los registros de activación del procedimiento `Modificar_Estudiante`.

Finalmente, la Figura 5 presenta el **Activation Record Instance (ARI)** correspondiente a la última llamada del procedimiento `Imprimir_Estudiante`. Al igual que en los casos anteriores, el ARI contiene los **enlaces estático y dinámico** que apuntan al ARI del procedimiento invocador (`Subprogramas_Ada`), junto con la **dirección de retorno** al mismo.

En la parte superior del registro se ubican los **parámetros por valor** del procedimiento, que contienen los datos del estudiante: `id`, `nombre` y `promedio`. El propósito de `Imprimir_Estudiante` es mostrar la información del estudiante ya modificada por el procedimiento anterior, antes de devolver el control a `Subprogramas_Ada`.

De esta forma se completa la secuencia de activaciones y retornos que ilustran el manejo de la **pila de activación** en las llamadas y finalización de procedimientos en Ada.

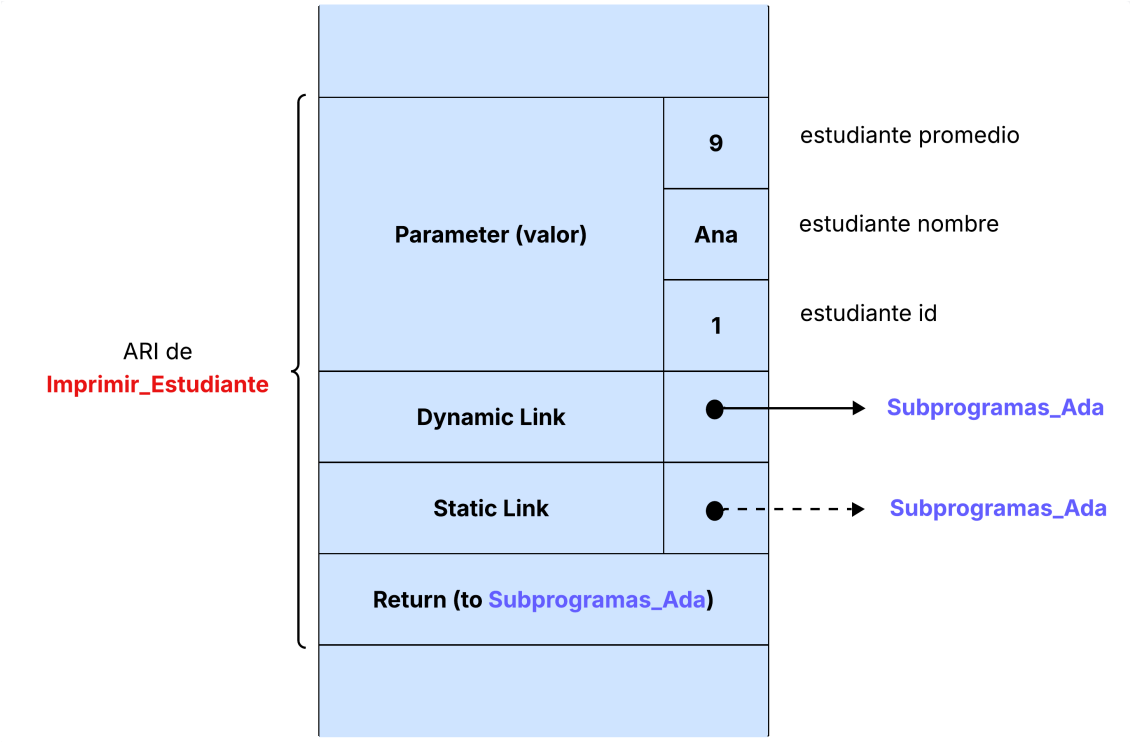


Figura 5: Pila de los registros de activación de la segunda llamada al procedimiento `Imprimir_Estudiante`.

3. Ejercicio 3

3.1. Static (Lexical) Scope

El **alcance estático** (o *lexical scope*) determina qué variables son visibles para un subprograma en tiempo de compilación, según dónde están declaradas en el código fuente.

Fue introducido por ALGOL 60 y adoptado por la mayoría de los lenguajes modernos (C, Java, Python, Ada, JavaScript, etc.).

Bajo este enfoque, tanto el compilador como el lector humano pueden determinar qué variable se está usando simplemente analizando el código, sin necesidad de ejecutarlo.

En lenguajes que permiten anidación de subprogramas, la búsqueda de una variable se realiza en el subprograma actual; si no se encuentra, se busca en el *padre estático* (es decir, el bloque donde fue definido ese subprograma), y así sucesivamente hasta llegar al nivel global. Si no se encuentra la variable, se produce un error.

Ejemplo conceptual

```
Main()
{
    x = 10 // global
    procedure f() {
        print(x)
    }
    procedure g() {
        x = 20
        f()
    }
    g()
}
```

Resultado con static scope: `f()` imprime `10`, porque la variable `x` que usa `f()` se determina en el entorno donde fue definida (el global), no desde donde fue llamada.

Lenguajes típicos: C, Java, Python, Ada, Scheme, JavaScript, Common Lisp.

3.2. Dynamic Scope

El **alcance dinámico** determina qué variables son visibles para un subprograma en tiempo de ejecución, según desde dónde fue llamada la función, no dónde fue definida.

En este modelo, la búsqueda de una variable se realiza recorriendo la pila de activación (*call stack*) en el orden de las llamadas activas. Por tanto, el valor que se usa puede cambiar dependiendo del contexto de la llamada.

Ejemplo conceptual (mismo código)

```
Main()
{
    x = 10 // global
    procedure f() {
        print(x)
    }
    procedure g() {
        x = 20
        f()
    }
    g()
}
```

Resultado con dynamic scope: `f()` imprime `20`, porque en tiempo de ejecución la llamada de `f()` ocurre dentro de `g()`, y `g()` tiene su propia `x = 20`.

Lenguajes típicos: versiones antiguas de Lisp, APL, SNOBOL4, y opcionalmente en Perl o Common Lisp.

Diferencia clave

Característica	Static Scope	Dynamic Scope
Determinado en	Compilación	Ejecución
Búsqueda de variables	Según el bloque de definición	Según la secuencia de llamadas
Ventaja	Predecible, más seguro	Flexible, adaptable
Lenguajes típicos	C, Java, Python, Ada	Lisp antiguo, APL, Perl (opcional)

4. Deep Binding vs Shallow Binding

Estos mecanismos se aplican cuando un subprograma se pasa como parámetro o se almacena como valor (por ejemplo, una función pasada a otra función o un *closure*). Determinan qué entorno de referencia usará el subprograma al ejecutarse.

4.1. Deep Binding

El **Deep Binding** establece que el entorno de referencia de un subprograma pasado como parámetro es el del lugar donde fue definido, no donde se lo llama. De esta forma, al ejecutarse, el subprograma accede a las variables del contexto de su definición, manteniendo la coherencia con el alcance estático, ya que el entorno se fija en el momento de su creación o definición y las referencias a variables no locales se resuelven según el contexto en el que el subprograma fue definido originalmente.

Ejemplo

```
Main()
{
    var x = 1 // global
    procedure P() {
        print(x)
    }

    procedure Q(f) {
        var x = 2
        f() // llamada de la funcion pasada como parametro
    }
    Q(P)
}
```

Comportamiento con Static Scope + Deep Binding: `P` fue definida en el entorno donde `x = 1`. Aunque se llame dentro de `Q`, recuerda su contexto original. **Salida:** 1.

4.2. Shallow Binding

El **Shallow Binding** indica que el entorno de referencia de un subprograma es el del punto de llamada, es decir, el contexto activo en el momento en que se ejecuta. En este modelo, el subprograma accede a las variables del subprograma que lo invocó, no de aquel donde fue definido, correspondiendo así al modelo de alcance dinámico, donde las variables no locales se resuelven según el contexto de ejecución actual.

Ejemplo (mismo código anterior)

```
Main()
{
    var x = 1 // global
    procedure P() {
        print(x)
    }

    procedure Q(f) {
        var x = 2
        f() // llamada de la funcion pasada como parametro
    }
    Q(P)
}
```

Comportamiento con Dynamic Scope + Shallow Binding: `P` no conserva el entorno donde fue definida. En el momento de ejecución, busca `x` en el entorno de la llamada (`Q`). **Salida:** 2.

Resumen comparativo

Concepto	Cuándo se fija el entorno	Modelo asociado	Acceso a variables
Deep Binding	Al definir la función	Static scope	Variables del contexto de definición
Shallow Binding	Al ejecutar la función	Dynamic scope	Variables del contexto de llamada